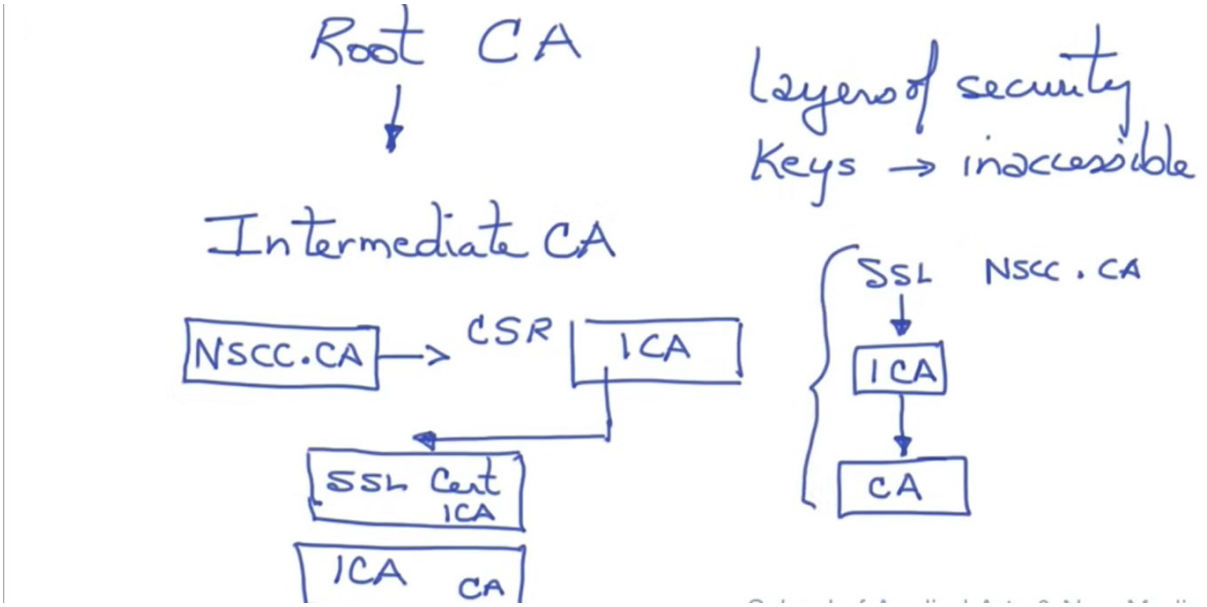
keywords like isDeviceRooted, root, rootUtils, /sys, /system, superuser **isDeviceRooted**

MobSF don't provide an options or environment for automated dynamic analysis but provide the semi-automated environment for dynamic analysis because Mobsf don't know what is the business login of the application such as what data provide to the displayed activity and how it will move to the next activity. So, it is the responsibility of the user to navigate each functionality of the android application once dynamic analysis starts.

If you perform dynamic analysis using MobSF and if your test application with android version 5.0 or higher then you will get more option with Frida Script.

Remove CA certificate:

Root CA: In cryptography and computer security, a root certificate is a public key certificate that identifies a root certificate authority. Root certificates are self-signed and form the basis of an X.509-based public key infrastructure. Or A Root CA is a Certificate Authority that owns one or more trusted roots. That means that they have roots in the trust stores of the major browsers. Intermediate CAs or Sub CAs are Certificate Authorities that issue off an intermediate root.



https://www.youtube.com/watch?v=heacxYUnFHA

Removing Root CA is responsible to intercept the traffic from the device. If you want then you can install or you can remove also

Exported Activity Tester: This functionality will test all the exported activity by opening each of the exported activity automatically in the AVD; and test each of it. During the testing, Mobsf also takes the screenshots of it for proof of concepts.

Start Activity Tester: Start activity tester is used to brute force non-exported activity. MobSF will test each of the non-exported activity and try to brute force each of it. During the brute force operation, it will also take the screen shots for PoC.

Take Screenshots: This functionality used to take the screen shots of AVD interface. During the auditing, if you feel that you would like to take the screen shots of that interface and also would like to add that screen shot as part of your report then you can use this functionality.

Logcat stream: This will show that logcat stream of the device.

Generate Report: This will stop all the operation and generate the report.

Dynamic analysis: Shows that activity status.

Error: Shows that log of the error.

Dynamic Analyzer

The MobSF will not do the dynamic analysis automatically but you need to visit the activity individually after selecting the Frida Scripts options.

Default Section

Default section have default loaded Frida scripts.

- API monitoring : This functionality check all the API at run time
- SSL Pinning Bypass: This functionality is responsible to bypass the SSL
  Explain SSL Pinning with simple codes:
  There are two major factors in an HTTPS connection, **a valid certificate** that **server presents during handshaking,** and a **cipher suite to be used for data encryption during transmission.** The certificate is the essential component and serves as a proof of identity of the server. The client will only trust the server if the server can provide a valid certificate that is signed by one of the **trusted Certificate Authorities** that come pre-installed in the client, otherwise, the connection will be aborted.
  An attacker can abuse this mechanism by **either install a malicious root CA certificate** to user devices so the client will trust all certificates that are signed by the attacker, or even worse, **compromised a CA completely**. Therefore, relying on the certificates received from servers alone cannot guarantee the **authenticity of the server,** and it is vulnerable to a potential man-in-the-middle attack.
  SSL Pinning is a technique that we use in the **client side to avoid man-in-the-middle attack** by **validating the server certificates again even after SSL handshaking. The developers embed (or pin) a list of trustful certificates to the client application during development**, and u**se them to compare against the server certificates during runtime**. If there is a mismatch between the server and the local copy of certificates, the connection will simply be disrupted, and no further user data will be even sent to that server. This enforcement ensures that the user devices are communicating only to the dedicated trustful servers.
  However, developers must take e**xtra caution in SSL Pinning**, when a pinned certificate is expired and the server has updated a new certificate, since the new certificate is definitely different from the pinned one that the client currently has, the clients will not trust the updated certificate and therefore terminate the connection, in which no further communications can be established between clients and servers, so the application is basically ‘bricked’. Therefore, to avoid such situation, it is always advisable to pin the future certificates in the client applications before release.

There are usually two ways we can achieve SSL Pinning in client applications. Pin either the whole certificate or its hashed public key. The hashed public key pinning is the preferred approach because the same private key can be used in signing the updated certificate, therefore we can save the trouble of pinning a new hashed public key for a new certificate, and reduce the risk of app 'bricking'.
Curtesy: https://zhangqichuan.medium.com/explain-ssl-pinning-with-simple-codes-eaee95b70507

- **Root detection**
  Nowadays many applications such as **financial, banking, payment wallet applications** do not work on the rooted device. **Pen testing requires root permission** to **install various tools to compromise the security of the application and it is very painful job for any pen tester if app does not work on the rooted device** that restrict the tester from performing various test cases.
  Curtesy: https://medium.com/@sarang6489/root-detection-bypass-by-manual-code-manipulation-5478858f4ad1
- **Debugger check By-pass:**
  Debugging is a highly effective way to **analyze runtime app behavior.** It allows the reverse engineer to step through the code, stop app execution at arbitrary points, inspect the state of variables, read and modify memory, and a lot more.
  Curtesy : https://mobile-security.gitbook.io/mobile-security-testing-guide/android-testing-guide/0x05j-testing-resiliency-against-reverse-engineering

Practical

- Keep default Frida script and click on the start instrumentation: move to AVD and also open the Frida script log which shows that output of Frida script.
- Frida Live script is also useful in the case when you write your own script and you would like to see the output.
- Now click on the different activity so MobSF will apply the various selected script while you navigate the functionality of the application.